

Distinguish between application software and system software

Application Software	System Software
1. Performs specific user tasks	1. Manages hardware and system resources
2. User-oriented	2. System-oriented
3. Runs on top of system software	3. Runs directly on hardware
4. Not essential for system startup	4. Essential for system startup
5. Installed as per user needs	5. Installed by default
6. Used by end users	6. Used by system and applications
7. Examples: Word, Excel, Browser	7. Examples: OS, Compiler, Driver

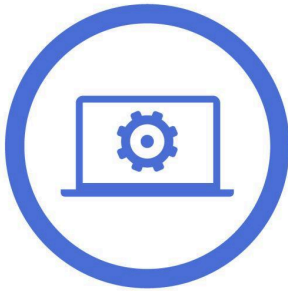
Explain various system softwares in detail.

System Software

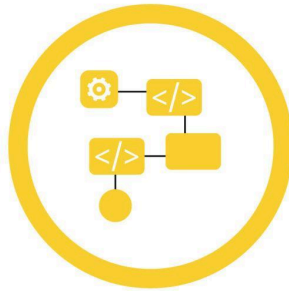
System software is a collection of programs that **control, manage, and coordinate computer hardware** and provide a **platform for application software** to run.

Types of System Software

3 Types of System Software



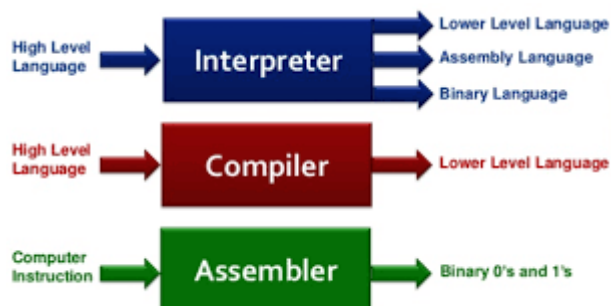
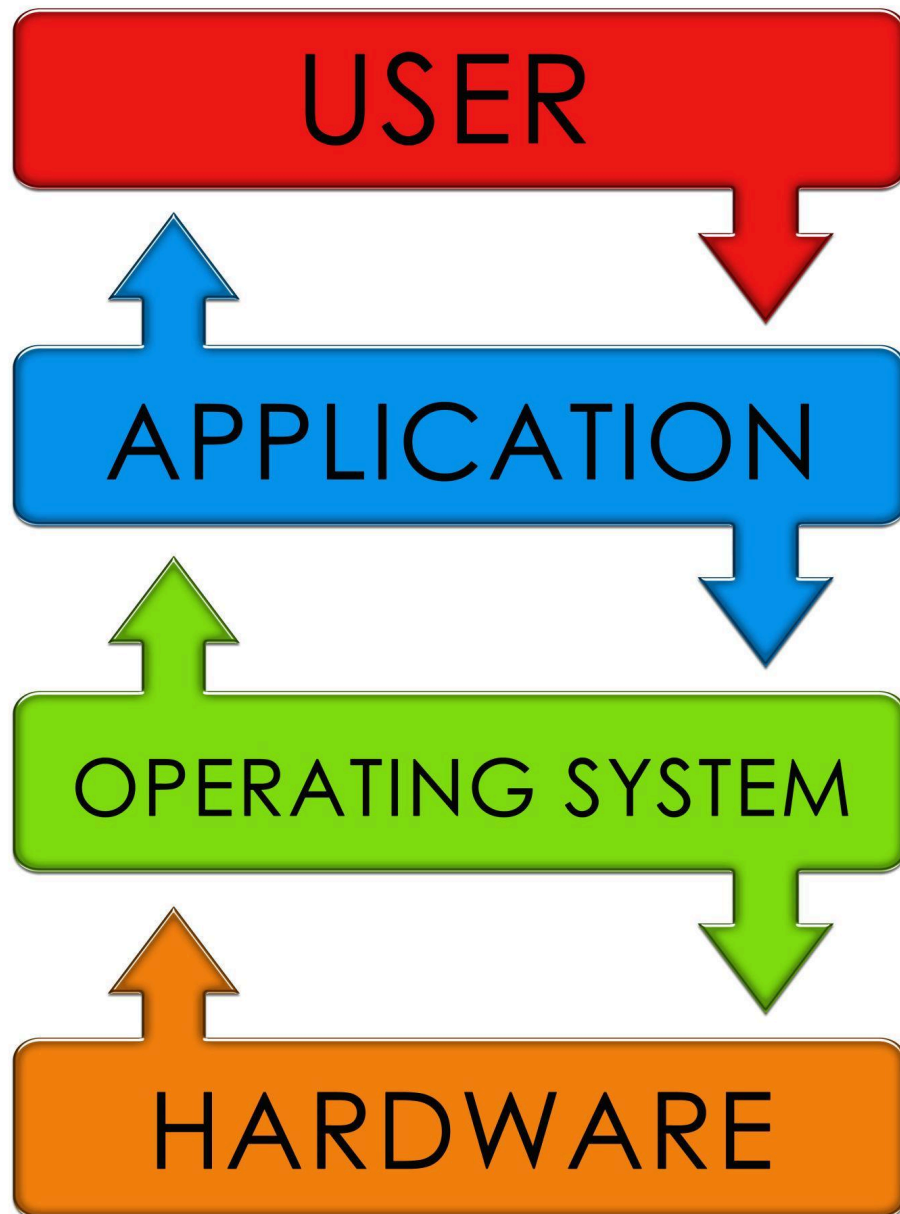
Operating
System



Language
Processor



Utility
Software

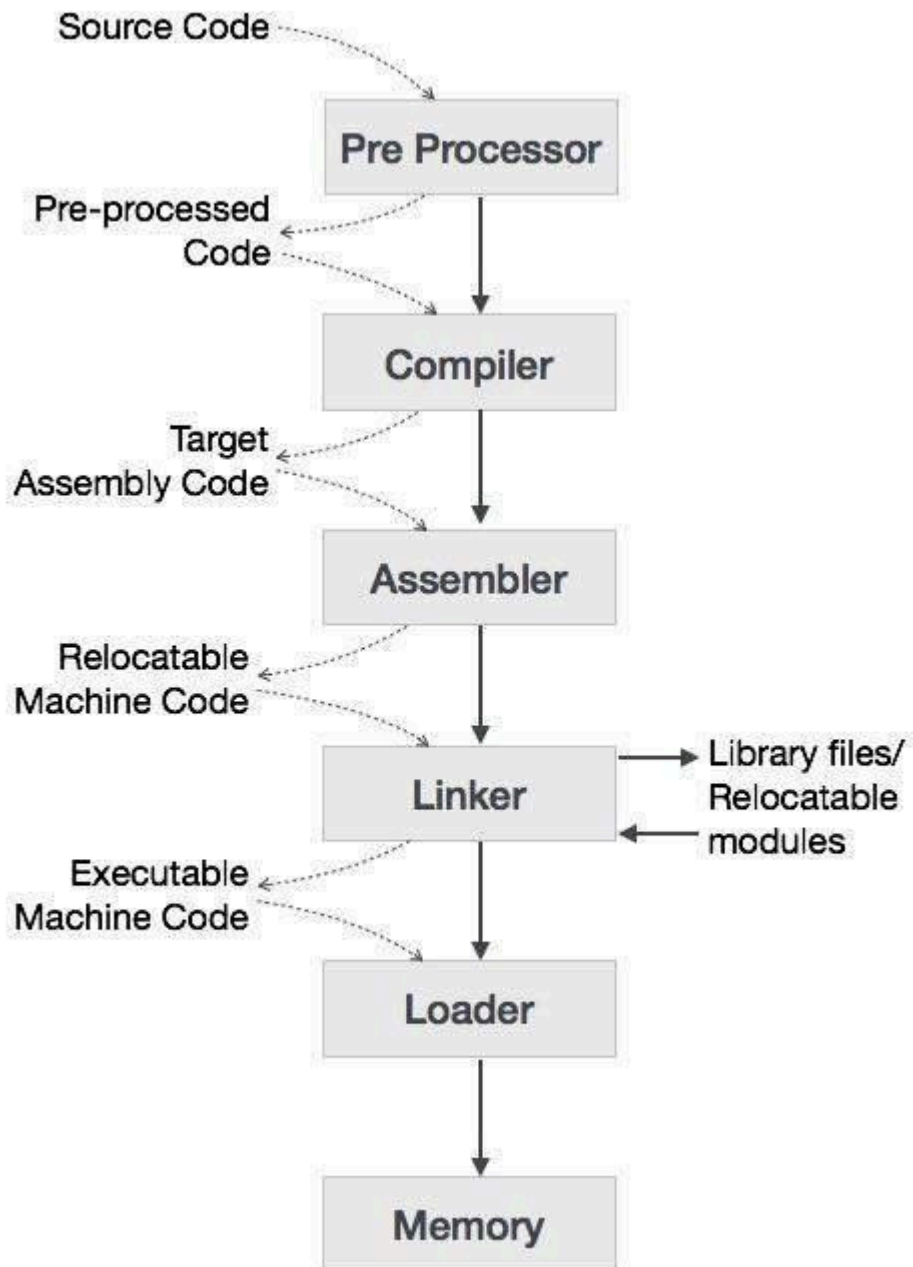


Difference Between Compiler and Assembler



High level → **Compiler** → Low level

Assembly Code → **Assembler** → Machine Code



1. Operating System (OS)

An **Operating System** is the most important system software. It acts as an **interface** between the user and the hardware.

Functions:

- Process management
- Memory management
- File management

- Device management
- Security and access control

Examples:

- Windows
- Linux
- macOS
- Android

 Without an OS, a computer cannot function.

2. Language Translators

Language translators convert programs written in **high-level or assembly language** into **machine language**.

(a) Compiler

- Translates the **entire program at once**
- Produces object code
- Faster execution after compilation

Example: C, C++

(b) Interpreter

- Translates and executes **line by line**
- No object code generated
- Easier debugging

Example: Python, JavaScript

(c) Assembler

- Converts **assembly language** into machine code
 - Used in system-level programming
-

3. Device Drivers

Device drivers are programs that allow the **operating system to communicate with hardware devices**.

Examples:

- Printer driver
- Keyboard driver
- Graphics card driver
- Sound driver

 Each hardware device requires a specific driver.

4. Linker

A **linker** combines **multiple object files** into a single executable file.

Functions:

- Resolves external references
- Combines program modules
- Produces final executable

 Used after compilation.

5. Loader

A **loader** loads the executable program into **main memory** and prepares it for execution.

Functions:

- Allocates memory
 - Relocates addresses
 - Starts program execution
-

6. Utility Programs

Utility programs help in **system maintenance and performance improvement**.

Examples:

- Antivirus software

- Disk cleanup tools
- Backup utilities
- File compression tools

📌 Utilities enhance system efficiency and security.

7. Firmware

Firmware is **permanent software** stored in **ROM or flash memory**.

Examples:

- BIOS
- UEFI

Functions:

- Initializes hardware during startup
 - Loads the operating system
-

Summary Table (Quick Revision)

System Software	Purpose
Operating System	Controls and manages system
Compiler	Translates high-level language
Interpreter	Executes line by line
Assembler	Translates assembly language
Device Driver	Controls hardware devices
Linker	Combines object files
Loader	Loads program into memory
Utilities	Maintains system
Firmware	Startup and hardware control

Explain the goals of system software

Goals of System Software

The main goal of **system software** is to **control and manage computer hardware efficiently** and to provide a **convenient environment** for application software and users.

1. Efficient Use of Hardware

- System software ensures optimal use of **CPU, memory, storage, and I/O devices**.
 - Prevents wastage of system resources.
-

2. Convenience for Users

- Provides an easy interface for users to interact with the computer.
 - Hides complex hardware details from the user.
-

3. Resource Management

- Manages system resources such as:
 - Processor
 - Memory
 - Files
 - Devices
 - Ensures fair and proper allocation of resources.
-

4. Program Execution Support

- Helps in loading, linking, and executing programs.
 - Provides services needed by application programs during execution.
-

5. Portability

- Allows programs to run on different hardware with minimal changes.
 - Achieved by hiding hardware-specific details.
-

6. Reliability and Error Handling

- Detects and handles system errors.
 - Prevents system crashes and improves stability.
-

7. Security and Protection

- Protects data and programs from unauthorized access.
 - Ensures controlled access to system resources.
-

8. System Performance Improvement

- Improves speed, efficiency, and overall system performance.
 - Optimizes scheduling and memory usage.
-